COSC 42: Distributed Computing

Problem 1. FCFS Mutex with Fetch-and-Add and Bounded Values.
Recall that the Ticket Lock from lecture requires a counter whose value grows unboundedly. The goal in this problem will be to design a lock with fetch-and-add that ensures that values held in memory are bounded to polynomials in n, and thereby only require $O(\log n)$ bits to store.
Design an n-process mutual exclusion lock that satisfies Mutex, Starvation Freedom, FCFS, and Bounded Exit. You may only use the read, write, and fetch-and-add shared-memory instructions in your algorithm. The algorithm must not require shared or local variables with more than $O(\log n)$ bits. You may assume that n is fixed and known, and that the processes are named $1, 2, \ldots, n$.

Problem 2. FCFS from Starvation Freedom .
Reduce n-process FCFS Mutex to n-process Starvation Free Mutex. That is, let $n \geq 2$ be any fixed and known integer, and let L be a solution to the mutual exclusion problem that satisfies Mutex and Starvation Freedom, construct by reduction a lock L' for n processors that satisfies Mutex, Starvation Freedom, and First-Come-First-Served (FCFS). Your reduction should only use the read and write shared-memory instructions (i.e., it should not use RMW instructions). You may assume that n is known and that the processes have names in the set $\{1, 2, \ldots, n\}$.

Problem 3. $O(1)$ **DSM RMR Mutex with Fetch-and-Add (Optional)**.
Read about the MCS Lock that uses the fetch-and-store instruction to achieve a lock satisfying FCFS with $O(1)$ RMR complexity in both the DSM and CC models.
Design an n-process mutual exclusion lock that satisfies Mutex, Starvation Freedom, FCFS, and Bounded Exit. You may only use the read, write, and fetch-and-add shared-memory instructions in your algorithm. Your algorithm must have $O(1)$ RMR Complexity on both DSM and CC machines. You may assume that n is fixed and known.